18/26

Ex. 6 Error detection and correction using Hamming code.

Aim:

To write a program to implement error detection and correction using the Hamming code.

Algorithm:

1. Start the program
2. Read the number of 4-bit data blocks (n) forming the input data stream.
3. For each 4-bit data block $D = d_1 d_2 d_3 d_4$, perform Hamming encoding.
4. a) Arrange 7 bits with parity bits using at positions 1, 2, and 4, data bits at position 3, 5, 6 and 7
   b) Compute the parity bits using even parity.
   $P_1 = d_1 \text{ XOR } d_2 \text{ XOR } d_4$ (Covers bit positions 1, 3, 5, 7)
   $P_2 = d_1 \text{ XOR } d_3 \text{ XOR } d_4$ (Covers bit position 2, 3, 6, 7)
   $P_4 = d_2 \text{ XOR } d_3 \text{ XOR } d_4$ (covers bit position 4, 5, 6, 7)
   c.) Form the 7-bit Hamming code word: $P_1, P_2, d_4 d_2$.
4. Simulate transmission by introducing a single bit
5. If the received code word
   $c_1 = P_1 \text{ XOR } d_1 \text{ XOR } d_2 \text{ XOR } d_4$
   $c_2 = P_2 \text{ XOR } d_1 \text{ XOR } d_3 \text{ XOR } d_4$
   $c_3 = P_4 \text{ XOR } d_2 \text{ XOR } d_3 \text{ XOR } d_4$
6. Compute Error position $= c_4 c_2 c_1$ (binary number formed from the check bits)
7. id Error position = 0, no error has occured Else, the bit at "Error position" is Inverted

(complemented) to correct the error.

8. Extract the original 4 data bits from the corrected 7-bit code word.

9. Repeat step 3 to 8 for all n-blocks in the data stream.

10. Stop the program.

Program

```
def encode(d):
    """ d: list of 4 bits [d1, d2, d3, d4] returns 7-bit
    Hamming code list positions (1-indexed): 1=P1 2=P2
    3=d1  4=P4  5=d2  6=d3  7=d4 """
    h[0] = 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1 * 1 + c2 * 2 + c4 * 4
    if error_pos != 0:
    print(f" Error detected at bit position :
                {error_pos} -> correcting ")
        h[error_pos - 1] ^= 1

    else:
        print("No error detected")
        return error_pos
```

output:

::: Hamming Code - Error Detection and Correction

::: Enter number of 4-bit data blocks in the stream : 2

-- block 1 --

Enter 4 data bits (space separated eg. 1011): 1011

Original Data Bits : 1011

Encoded Hamming code (7 bits): 0 1 1 0 1 1

Enter bit position to introduce error (1-7, 0=no error): 3

Transmitted code (with error induced at pos 3): 0 1 0 0 0 1 1

Received code : 0 1 0 0 0 1 1

Checking for errors....

Error detected at bit position : 3 -> Correcting...

Corrected code : 0 1 1 0 0 1 1

Recovered Data Bits : 1 0 1 1

```
def to_str(bits):
    return "".join(str(b) for b in bits)
def main()
print ("=== Hamming code- Error- detection and
        correction ===\n")
    n= int ( input ("Enter number of 4 bits data
        blocks in the stream: "))
print (f" original Data Bits : {to_str(d)}")
print (f" Encoded Hamming code (7 bits): {to_str(h)}")
pos = int (input (" Enter bit position to introduce
        Error (1-7, 0=no error): "))
if 1 <= pos <=7
    h[pos -1] ^= 1
print (f" Transmitted code (no error induced):
        {to_str(h)}")
print (f" Received code : {to_ str(h)}")
print ("Checking for error...")
detect_and_ correct(h)
print (f" corrected code : {to_str(h)}")
 rd = [h[2], h[4], h[5], h[6]]
 print (f" Recovered Data Bits : {to_str(rd)}")
 if __name__ == "__main__":
 main():
```

95

Result:

The program to implement error detection and correction using the Hamming code concept was successfully written in python, executed and verified usage input data stream.

6/8